

International Islamic University Chittagong



Project Report

Topic: Multiple 3D Games

Course Title: Computer Graphics Lab

Course Code: CSE-4742

<u>Submitted By</u>	<u>Submitted To</u>
Name: Sabrina Afroz Nitu Id: C223241 Section: 7BF Semester: 7 th Dept of CSE, IIUC	Sultana Tasnim Jahan Lecturer Dept of CSE, IIUC

Marks:	Signature:
---------------	-------------------

1. Introduction:

Computer graphics plays an important role in modern game development. This project, Enhanced 3D Multi-Game Collection, is a graphics-based gaming application developed using C++ with OpenGL and GLUT. The system integrates multiple classic games such as Snake, Pong, Shooter, and Tetris into a single 3D environment. Each game includes real-time rendering, sound effects, particle effects, lighting, and camera control, which enhance the overall user experience. The project demonstrates practical use of computer graphics concepts, real-time interaction, and game logic implementation.

2. Objectives:

The main objectives of this project are-

- To develop a multi-game platform using OpenGL and GLUT
- To implement 3D graphics rendering with lighting and depth
- To design interactive gameplay with keyboard controls
- To integrate sound effects for better user experience
- To apply particle systems for explosion and visual effects
- To understand game state management and real-time updates

3.Tools and Technologies:

- Programming Language: C++
- Graphics Library: OpenGL
- Utility Toolkit: GLUT
- IDE: Code::Blocks
- Operating System: Windows

4. Main Features of the Project

- 🎮 Multiple games in one application
- 📦 3D graphical visualization
- ⌨️ Keyboard-based controls
- 🧠 Collision detection system
- 📊 Score tracking and game over detection
- 🖥️ Menu-driven interface
- ⚡ Real-time game updates

5. Methodology



6.Code:

// Enhanced 3D Multi-Game Collection with Tetris + Sound Effects - OpenGL
GLUT

// Compile: g++ -o game game.cpp -lGL -lGLU -lglut -lm

```
#include <GL/glut.h>
#include <GL/gl.h>
#include <GL/glu.h>
#include <cmath>
#include <vector>
#include <cstdlib>
#include <ctime>
#include <string>
#include <iostream>
#include <algorithm>
#include <cstring>
#include <chrono>
#include <thread>
```

```
using namespace std;
```

```
// Cross-platform sleep function
void sleepMs(int milliseconds) {
    std::this_thread::sleep_for(std::chrono::milliseconds(milliseconds));
}
```

```
// Sound system using beep (Linux) or system beep (cross-platform)
void playBeep(int frequency, int duration) {
    #ifdef __linux__
        // Use beep command on Linux for better sound
        char cmd[100];
        sprintf(cmd, "beep -f %d -l %d 2>/dev/null &", frequency, duration);
        system(cmd);
    #else
        // Simple beep for other systems
        cout << "\a" << flush;
    #endif
}
```

```
void playSoundEffect(const char* type) {
```

```

if (strcmp(type, "menu_select") == 0) {
    playBeep(800, 100);
} else if (strcmp(type, "game_start") == 0) {
    playBeep(600, 80);
    sleepMs(90);
    playBeep(800, 80);
    sleepMs(90);
    playBeep(1000, 120);
} else if (strcmp(type, "score") == 0) {
    playBeep(1200, 60);
    sleepMs(70);
    playBeep(1400, 80);
} else if (strcmp(type, "hit") == 0) {
    playBeep(300, 100);
} else if (strcmp(type, "explosion") == 0) {
    playBeep(200, 50);
    sleepMs(60);
    playBeep(150, 50);
    sleepMs(60);
    playBeep(100, 80);
} else if (strcmp(type, "game_over") == 0) {
    playBeep(800, 100);
    sleepMs(110);
    playBeep(600, 100);
    sleepMs(110);
    playBeep(400, 100);
    sleepMs(110);
    playBeep(200, 200);
} else if (strcmp(type, "pause") == 0) {
    playBeep(700, 100);
    sleepMs(110);
    playBeep(500, 150);
} else if (strcmp(type, "line_clear") == 0) {
    playBeep(1000, 60);
    sleepMs(70);
    playBeep(1200, 60);
    sleepMs(70);
    playBeep(1400, 60);
    sleepMs(70);
    playBeep(1600, 100);
}

```

```

    } else if (strcmp(type, "tetris_drop") == 0) {
        playBeep(400, 80);
    } else if (strcmp(type, "shoot") == 0) {
        playBeep(1500, 40);
        sleepMs(50);
        playBeep(1800, 30);
    } else if (strcmp(type, "bounce") == 0) {
        playBeep(900, 60);
    } else if (strcmp(type, "food_eat") == 0) {
        playBeep(1100, 70);
        sleepMs(80);
        playBeep(1300, 90);
    }
}

// Game States
enum GameMode { MENU, INSTRUCTIONS, SNAKE, PONG, SHOOTER,
TETRIS, PAUSED, GAME_OVER };
GameMode currentGame = MENU;
GameMode previousGame = MENU;
int score = 0;
int highScores[4] = {0, 0, 0, 0};
int lives = 3;
float gameTime = 0;

// Window dimensions
int windowHeight = 1000;
int windowHeight = 700;

// Camera controls
float cameraRotation = 0;
float cameraDistance = 20.0f;

// Particle system
struct Particle {
    float x, y, z;
    float vx, vy, vz;
    float life;
    float r, g, b;
};

```

```

vector<Particle> particles;

// Snake Game Variables
struct SnakeSegment {
    float x, z;
};
vector<SnakeSegment> snake;
float foodX = 0, foodZ = 0;
int snakeDir = 0;
int nextDir = 0;
int moveCounter = 0;

// Pong Game Variables
float playerPaddleY = 0;
float aiPaddleY = 0;
float ballX = 0, ballY = 0;
float ballVelX = 0.05f, ballVelY = 0.03f;
bool upPressed = false, downPressed = false;
int playerScore = 0, aiScore = 0;

// Shooter Game Variables
float playerX = 0;
struct Enemy {
    float x, z;
    float rotation;
};
struct Bullet {
    float x, z;
};
vector<Enemy> enemies;
vector<Bullet> bullets;
int spawnCounter = 0;
bool leftPressed = false, rightPressed = false;

// Tetris Game Variables
const int TETRIS_ROWS = 20;
const int TETRIS_COLS = 10;
const float TETRIS_BLOCK_SIZE = 0.4f;
int tetrisBoard[20][10] = {0};
struct Point { int x, y; };

```

```

Point tetrisCurrent[4], tetrisTemp[4];
int tetrisCurrentType, tetrisNextType;
int tetrisCurrentColor, tetrisNextColor;
bool tetrisRotateFlag = false;
int tetrisDx = 0;
int tetrisMoveCounter = 0;
int tetrisLevel = 1;
int tetrisLinesCleared = 0;

int tetrisFigures[7][4] = {
    {1, 3, 5, 7}, {2, 4, 5, 7}, {3, 5, 4, 6}, {3, 5, 4, 7},
    {2, 3, 5, 7}, {3, 5, 7, 6}, {2, 3, 4, 5}
};

float tetrisColors[8][3] = {
    {0.2, 0.2, 0.2}, // Black
    {0.1, 0.95, 0.95}, // Bright Cyan
    {0.95, 0.1, 0.1}, // Bright Red
    {0.1, 0.95, 0.1}, // Bright Green
    {0.3, 0.4, 0.95}, // Bright Blue
    {0.95, 0.95, 0.1}, // Bright Yellow
    {0.95, 0.1, 0.95}, // Bright Magenta
    {0.95, 0.6, 0.1} // Bright Orange
};

// Function declarations
void init();
void display();
void reshape(int w, int h);
void keyboard(unsigned char key, int x, int y);
void keyboardUp(unsigned char key, int x, int y);
void specialKeys(int key, int x, int y);
void specialKeysUp(int key, int x, int y);
void timer(int value);
void drawText(float x, float y, const char* text, void* font =
GLUT_BITMAP_HELVETICA_18);
void drawText3D(float x, float y, float z, const char* text);
void drawMenu();
void drawInstructions();
void drawPauseMenu();

```

```

void drawGameOver();
void createExplosion(float x, float y, float z);
void updateParticles();
void drawParticles();
void drawGround();
void initSnakeGame();
void updateSnakeGame();
void drawSnakeGame();
void initPongGame();
void updatePongGame();
void drawPongGame();
void initShooterGame();
void updateShooterGame();
void drawShooterGame();
void initTetrisGame();
void updateTetrisGame();
void drawTetrisGame();
void placeFood();

// Initialize OpenGL with better lighting
void init() {
    glClearColor(0.05f, 0.05f, 0.15f, 1.0f);
    glEnable(GL_DEPTH_TEST);
    glEnable(GL_LIGHTING);
    glEnable(GL_LIGHT0);
    glEnable(GL_LIGHT1);
    glEnable(GL_COLOR_MATERIAL);
    glColorMaterial(GL_FRONT_AND_BACK,
GL_AMBIENT_AND_DIFFUSE);
    glEnable(GL_NORMALIZE);
    glShadeModel(GL_SMOOTH);

    // Main light
    GLfloat light0_pos[] = {0.0f, 15.0f, 10.0f, 1.0f};
    GLfloat light0_ambient[] = {0.3f, 0.3f, 0.4f, 1.0f};
    GLfloat light0_diffuse[] = {1.0f, 1.0f, 1.0f, 1.0f};
    GLfloat light0_specular[] = {1.0f, 1.0f, 1.0f, 1.0f};

    glLightfv(GL_LIGHT0, GL_POSITION, light0_pos);
    glLightfv(GL_LIGHT0, GL_AMBIENT, light0_ambient);

```

```

glLightfv(GL_LIGHT0, GL_DIFFUSE, light0_diffuse);
glLightfv(GL_LIGHT0, GL_SPECULAR, light0_specular);

// Fill light
GLfloat light1_pos[] = {-5.0f, 5.0f, -5.0f, 1.0f};
GLfloat light1_ambient[] = {0.2f, 0.2f, 0.3f, 1.0f};
glLightfv(GL_LIGHT1, GL_POSITION, light1_pos);
glLightfv(GL_LIGHT1, GL_AMBIENT, light1_ambient);

// Material properties
GLfloat mat_specular[] = {1.0f, 1.0f, 1.0f, 1.0f};
GLfloat mat_shininess[] = {50.0f};
glMaterialfv(GL_FRONT, GL_SPECULAR, mat_specular);
glMaterialfv(GL_FRONT, GL_SHININESS, mat_shininess);

srand(time(0));
}

// Enhanced text drawing
void drawText(float x, float y, const char* text, void* font) {
    glDisable(GL_LIGHTING);
    glMatrixMode(GL_PROJECTION);
    glPushMatrix();
    glLoadIdentity();
    gluOrtho2D(0, windowWidth, 0, windowHeight);
    glMatrixMode(GL_MODELVIEW);
    glPushMatrix();
    glLoadIdentity();

    glColor3f(1.0f, 1.0f, 1.0f);
    glRasterPos2f(x, y);
    for (const char* c = text; *c != '\0'; c++) {
        glutBitmapCharacter(font, *c);
    }

    glPopMatrix();
    glMatrixMode(GL_PROJECTION);
    glPopMatrix();
    glMatrixMode(GL_MODELVIEW);
    glEnable(GL_LIGHTING);
}

```

```

}

void drawText3D(float x, float y, float z, const char* text) {
    glDisable(GL_LIGHTING);
    glColor3f(1.0f, 1.0f, 1.0f);
    glRasterPos3f(x, y, z);
    for (const char* c = text; *c != '\0'; c++) {
        glutBitmapCharacter(GLUT_BITMAP_HELVETICA_18, *c);
    }
    glEnable(GL_LIGHTING);
}

```

// Particle system

```

void createExplosion(float x, float y, float z) {
    for (int i = 0; i < 30; i++) {
        Particle p;
        p.x = x;
        p.y = y;
        p.z = z;
        float angle1 = (rand() % 360) * 3.14159f / 180.0f;
        float angle2 = (rand() % 360) * 3.14159f / 180.0f;
        float speed = 0.1f + (rand() % 100) / 500.0f;
        p.vx = cos(angle1) * sin(angle2) * speed;
        p.vy = sin(angle1) * speed;
        p.vz = cos(angle1) * cos(angle2) * speed;
        p.life = 1.0f;
        p.r = 1.0f;
        p.g = 0.5f + (rand() % 50) / 100.0f;
        p.b = 0.0f;
        particles.push_back(p);
    }
}

```

```

void updateParticles() {
    for (int i = particles.size() - 1; i >= 0; i--) {
        particles[i].x += particles[i].vx;
        particles[i].y += particles[i].vy;
        particles[i].z += particles[i].vz;
        particles[i].vy -= 0.005f; // gravity
        particles[i].life -= 0.02f;
    }
}

```

```

        if (particles[i].life <= 0) {
            particles.erase(particles.begin() + i);
        }
    }
}

void drawParticles() {
    glDisable(GL_LIGHTING);
    glPointSize(4.0f);
    glBegin(GL_POINTS);
    for (int i = 0; i < particles.size(); i++) {
        glColor4f(particles[i].r, particles[i].g, particles[i].b, particles[i].life);
        glVertex3f(particles[i].x, particles[i].y, particles[i].z);
    }
    glEnd();
    glEnable(GL_LIGHTING);
}

// Draw enhanced ground
void drawGround() {
    glDisable(GL_LIGHTING);
    glBegin(GL_QUADS);
    for (int i = -15; i < 15; i++) {
        for (int j = -15; j < 15; j++) {
            if ((i + j) % 2 == 0)
                glColor3f(0.1f, 0.15f, 0.2f);
            else
                glColor3f(0.15f, 0.2f, 0.25f);

            glVertex3f(i, -0.5f, j);
            glVertex3f(i + 1, -0.5f, j);
            glVertex3f(i + 1, -0.5f, j + 1);
            glVertex3f(i, -0.5f, j + 1);
        }
    }
    glEnd();
    glEnable(GL_LIGHTING);
}

```

```

// Enhanced menu
void drawMenu() {
    glDisable(GL_LIGHTING);
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    // Animated background
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(45.0, (double>windowWidth / windowHeight, 1.0, 100.0);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
    gluLookAt(0, 8, 15, 0, 0, 0, 0, 1, 0);

    glEnable(GL_LIGHTING);
    cameraRotation += 0.5f;

    // Rotating preview objects
    for (int i = 0; i < 4; i++) {
        glPushMatrix();
        glRotatef(cameraRotation + i * 90, 0, 1, 0);
        glTranslatef(5, 0, 0);
        glRotatef(cameraRotation * 2, 1, 1, 0);

        if (i == 0) glColor3f(0.0f, 1.0f, 0.0f);
        else if (i == 1) glColor3f(0.0f, 1.0f, 1.0f);
        else if (i == 2) glColor3f(1.0f, 0.0f, 0.0f);
        else glColor3f(1.0f, 1.0f, 0.0f);

        glutSolidCube(1.5f);
        glPopMatrix();
    }

    // Menu text
    drawText(320, 650, "ENHANCED 3D GAME COLLECTION",
    GLUT_BITMAP_TIMES_ROMAN_24);
    drawText(400, 600, "Choose Your Game", GLUT_BITMAP_HELVETICA_18);

    drawText(250, 500, "Press 1: 3D SNAKE GAME",
    GLUT_BITMAP_HELVETICA_18);
    char snake_score[50];

```

```

    sprintf(snake_score, "High Score: %d", highScores[0]);
    drawText(270, 470, snake_score, GLUT_BITMAP_HELVETICA_12);

    drawText(250, 410, "Press 2: 3D PONG GAME",
    GLUT_BITMAP_HELVETICA_18);
    char pong_score[50];
    sprintf(pong_score, "High Score: %d", highScores[1]);
    drawText(270, 380, pong_score, GLUT_BITMAP_HELVETICA_12);

    drawText(250, 320, "Press 3: 3D SHOOTER GAME",
    GLUT_BITMAP_HELVETICA_18);
    char shooter_score[50];
    sprintf(shooter_score, "High Score: %d", highScores[2]);
    drawText(270, 290, shooter_score, GLUT_BITMAP_HELVETICA_12);

    drawText(250, 230, "Press 4: 3D TETRIS GAME",
    GLUT_BITMAP_HELVETICA_18);
    char tetris_score[50];
    sprintf(tetris_score, "High Score: %d", highScores[3]);
    drawText(270, 200, tetris_score, GLUT_BITMAP_HELVETICA_12);

    drawText(350, 140, "Press I for Instructions",
    GLUT_BITMAP_HELVETICA_12);
    drawText(380, 100, "Press ESC to quit", GLUT_BITMAP_HELVETICA_12);

    glutSwapBuffers();
}

// Instructions screen
void drawInstructions() {
    glDisable(GL_LIGHTING);
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    drawText(400, 680, "INSTRUCTIONS",
    GLUT_BITMAP_TIMES_ROMAN_24);

    drawText(150, 630, "SNAKE GAME:", GLUT_BITMAP_HELVETICA_18);
    drawText(170, 605, "- Use Arrow Keys to move",
    GLUT_BITMAP_HELVETICA_12);
    drawText(170, 585, "- Eat red food to grow",

```

```
GLUT_BITMAP_HELVETICA_12);
```

```
    drawText(150, 540, "PONG GAME:", GLUT_BITMAP_HELVETICA_18);  
    drawText(170, 515, "- Use Up/Down arrows to move paddle",  
GLUT_BITMAP_HELVETICA_12);  
    drawText(170, 495, "- Don't let ball pass your paddle",  
GLUT_BITMAP_HELVETICA_12);
```

```
    drawText(150, 450, "SHOOTER GAME:",  
GLUT_BITMAP_HELVETICA_18);  
    drawText(170, 425, "- Use Left/Right arrows to move ship",  
GLUT_BITMAP_HELVETICA_12);  
    drawText(170, 405, "- Press SPACE BAR to shoot bullets",  
GLUT_BITMAP_HELVETICA_12);  
    drawText(170, 385, "- Destroy red cube enemies",  
GLUT_BITMAP_HELVETICA_12);
```

```
    drawText(150, 340, "TETRIS GAME:", GLUT_BITMAP_HELVETICA_18);  
    drawText(170, 315, "- Use Left/Right arrows to move piece",  
GLUT_BITMAP_HELVETICA_12);  
    drawText(170, 295, "- Press Up arrow to rotate piece",  
GLUT_BITMAP_HELVETICA_12);  
    drawText(170, 275, "- Press Down arrow for fast drop",  
GLUT_BITMAP_HELVETICA_12);  
    drawText(170, 255, "- Press SPACE for instant drop",  
GLUT_BITMAP_HELVETICA_12);  
    drawText(170, 235, "- Complete lines to score points",  
GLUT_BITMAP_HELVETICA_12);
```

```
    drawText(150, 190, "GENERAL CONTROLS:",  
GLUT_BITMAP_HELVETICA_18);  
    drawText(170, 145, "- Press ESC to return to menu",  
GLUT_BITMAP_HELVETICA_12);
```

```
    drawText(320, 90, "Press ENTER to return to menu",  
GLUT_BITMAP_HELVETICA_18);
```

```
    glutSwapBuffers();  
}
```

```

// Pause menu
void drawPauseMenu() {
    glDisable(GL_LIGHTING);
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    char scoreText[50];
    sprintf(scoreText, "Current Score: %d", score);
    drawText(400, 320, scoreText, GLUT_BITMAP_HELVETICA_18);

    if (previousGame == SHOOTER) {
        char livesText[50];
        sprintf(livesText, "Lives Remaining: %d", lives);
        drawText(400, 280, livesText, GLUT_BITMAP_HELVETICA_18);
    }

    drawText(350, 170, "Press ESC for main menu",
    GLUT_BITMAP_HELVETICA_18);

    glutSwapBuffers();
}

// Enhanced game over
void drawGameOver() {
    glDisable(GL_LIGHTING);
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    drawText(380, 450, "GAME OVER!", GLUT_BITMAP_TIMES_ROMAN_24);

    char scoreText[50];
    sprintf(scoreText, "Final Score: %d", score);
    drawText(400, 350, scoreText, GLUT_BITMAP_HELVETICA_18);

    char highScoreText[50];
    int gameIndex = (previousGame == SNAKE) ? 0 : (previousGame == PONG) ?
    1 : (previousGame == SHOOTER) ? 2 : 3;
    sprintf(highScoreText, "High Score: %d", highScores[gameIndex]);
    drawText(400, 300, highScoreText, GLUT_BITMAP_HELVETICA_18);

    drawText(300, 200, "Press ENTER to return to menu",

```

```

GLUT_BITMAP_HELVETICA_18);

    glutSwapBuffers();
}

// Snake Game Functions
void placeFood() {
    foodX = (rand() % 18) - 9.0f;
    foodZ = (rand() % 18) - 9.0f;
}

void initSnakeGame() {
    snake.clear();
    SnakeSegment head = {0, 0};
    snake.push_back(head);
    snakeDir = 0;
    nextDir = 0;
    moveCounter = 0;
    score = 0;
    lives = 3;
    placeFood();
}

void updateSnakeGame() {
    moveCounter++;
    if (moveCounter < 15) return;
    moveCounter = 0;

    snakeDir = nextDir;

    float newX = snake[0].x;
    float newZ = snake[0].z;

    if (snakeDir == 0) newX += 1;
    else if (snakeDir == 1) newZ -= 1;
    else if (snakeDir == 2) newX -= 1;
    else if (snakeDir == 3) newZ += 1;

    if (newX < -9 || newX > 9 || newZ < -9 || newZ > 9) {
        if (score > highScores[0]) highScores[0] = score;
    }
}

```

```

        previousGame = SNAKE;
        currentGame = GAME_OVER;
        playSoundEffect("game_over");
        createExplosion(newX, 1, newZ);
        return;
    }

    for (int i = 0; i < snake.size(); i++) {
        if (snake[i].x == newX && snake[i].z == newZ) {
            if (score > highScores[0]) highScores[0] = score;
            previousGame = SNAKE;
            currentGame = GAME_OVER;
            playSoundEffect("game_over");
            createExplosion(newX, 1, newZ);
            return;
        }
    }

    if (fabs(newX - foodX) < 1 && fabs(newZ - foodZ) < 1) {
        score += 10;
        placeFood();
        playSoundEffect("food_eat");
        createExplosion(foodX, 1, foodZ);
        SnakeSegment tail = snake[snake.size() - 1];
        snake.push_back(tail);
    }

    for (int i = snake.size() - 1; i > 0; i--) {
        snake[i] = snake[i - 1];
    }
    snake[0].x = newX;
    snake[0].z = newZ;
}

void drawSnakeGame() {
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(45.0, (double>windowWidth / windowHeight, 1.0, 100.0);

```

```

glMatrixMode(GL_MODELVIEW);
glLoadIdentity();
gluLookAt(0, 18, 18, 0, 0, 0, 0, 1, 0);

glEnable(GL_LIGHTING);
drawGround();

// Grid walls
glDisable(GL_LIGHTING);
glColor3f(0.2f, 0.4f, 0.6f);
glLineWidth(2.0f);
glBegin(GL_LINE_LOOP);
glVertex3f(-10, 0, -10);
glVertex3f(10, 0, -10);
glVertex3f(10, 0, 10);
glVertex3f(-10, 0, 10);
glEnd();
glEnable(GL_LIGHTING);

// Snake
for (int i = 0; i < snake.size(); i++) {
    glPushMatrix();
    glTranslatef(snake[i].x, 0.5f, snake[i].z);

    if (i == 0) {
        glColor3f(0.0f, 1.0f, 0.0f);
        glutSolidSphere(0.5f, 16, 16);

        glDisable(GL_LIGHTING);
        glColor3f(1.0f, 1.0f, 1.0f);
        glPushMatrix();
        glTranslatef(0.2f, 0.2f, 0.4f);
        glutSolidSphere(0.1f, 8, 8);
        glPopMatrix();
        glPushMatrix();
        glTranslatef(-0.2f, 0.2f, 0.4f);
        glutSolidSphere(0.1f, 8, 8);
        glPopMatrix();
        glEnable(GL_LIGHTING);
    } else {

```

```

        float brightness = 0.7f - (i * 0.02f);
        if (brightness < 0.3f) brightness = 0.3f;
        glColor3f(0.0f, brightness, 0.0f);
        glutSolidSphere(0.45f, 12, 12);
    }

    glPopMatrix();

    if (i < snake.size() - 1) {
        glDisable(GL_LIGHTING);
        glColor3f(0.0f, 0.6f, 0.0f);
        glLineWidth(8.0f);
        glBegin(GL_LINES);
        glVertex3f(snake[i].x, 0.5f, snake[i].z);
        glVertex3f(snake[i+1].x, 0.5f, snake[i+1].z);
        glEnd();
        glEnable(GL_LIGHTING);
    }
}

// Food
glPushMatrix();
glTranslatef(foodX, 0.5f, foodZ);
float pulse = 0.9f + 0.3f * sin(gameTime * 5);
glScalef(pulse, pulse, pulse);
glColor3f(1.0f, 0.0f, 0.0f);
glutSolidSphere(0.4f, 16, 16);

glDisable(GL_LIGHTING);
glColor3f(1.0f, 0.5f, 0.0f);
glutWireSphere(0.5f, 12, 12);
glEnable(GL_LIGHTING);
glPopMatrix();

drawParticles();

char scoreText[50];
sprintf(scoreText, "Score: %d", score);
drawText(10, windowHeight - 30, scoreText);
drawText(10, 30, "Arrow Keys | ESC:Menu");

```

```

    glutSwapBuffers();
}

// Pong Game Functions
void initPongGame() {
    playerPaddleY = 0;
    aiPaddleY = 0;
    ballX = 0;
    ballY = 0;
    ballVelX = 0.05f;
    ballVelY = 0.03f;
    score = 0;
    playerScore = 0;
    aiScore = 0;
}

void updatePongGame() {
    if (upPressed) playerPaddleY += 0.15f;
    if (downPressed) playerPaddleY -= 0.15f;
    if (playerPaddleY > 5.5f) playerPaddleY = 5.5f;
    if (playerPaddleY < -5.5f) playerPaddleY = -5.5f;

    if (ballY > aiPaddleY + 0.5f) aiPaddleY += 0.08f;
    else if (ballY < aiPaddleY - 0.5f) aiPaddleY -= 0.08f;
    if (aiPaddleY > 5.5f) aiPaddleY = 5.5f;
    if (aiPaddleY < -5.5f) aiPaddleY = -5.5f;

    ballX += ballVelX;
    ballY += ballVelY;

    // Top and bottom wall collision
    if (ballY > 6.5f || ballY < -6.5f) {
        ballVelY *= -1;
        playSoundEffect("bounce");
        createExplosion(ballX, ballY, 0);
    }

    // Player paddle collision (left side)
    if (ballX < -8.5f && ballX > -9.5f && fabs(ballY - playerPaddleY) < 1.8f) {

```

```

    ballVelX = fabs(ballVelX) * 1.1f; // Make sure it goes right
    score++;
    playerScore++;
    playSoundEffect("hit");
    createExplosion(ballX, ballY, 0);
}

// AI paddle collision (right side)
if (ballX > 8.5f && ballX < 9.5f && fabs(ballY - aiPaddleY) < 1.8f) {
    ballVelX = -fabs(ballVelX) * 1.1f; // Make sure it goes left
    playSoundEffect("hit");
}

// Player loses (ball goes past left paddle)
if (ballX < -10) {
    aiScore++;
    if (score > highScores[1]) highScores[1] = score;
    previousGame = PONG;
    currentGame = GAME_OVER;
    playSoundEffect("game_over");
}

// AI loses (ball goes past right paddle)
if (ballX > 10) {
    score += 10;
    playerScore++;
    // Reset ball
    ballX = 0;
    ballY = 0;
    ballVelX = -0.05f;
    ballVelY = 0.03f;
    playSoundEffect("score");
}
}

void drawPongGame() {
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();

```

```

gluPerspective(45.0, (double>windowWidth / windowHeight, 1.0, 100.0);
glMatrixMode(GL_MODELVIEW);
glLoadIdentity();
gluLookAt(0, 5, 22, 0, 0, 0, 0, 1, 0);

glEnable(GL_LIGHTING);

// Draw floor below the playing field
glPushMatrix();
glTranslatef(0, -8, 0);
glDisable(GL_LIGHTING);
glBegin(GL_QUADS);
for (int i = -15; i < 15; i++) {
    for (int j = -15; j < 15; j++) {
        if ((i + j) % 2 == 0)
            glColor3f(0.1f, 0.15f, 0.2f);
        else
            glColor3f(0.15f, 0.2f, 0.25f);

        glVertex3f(i, 0, j);
        glVertex3f(i + 1, 0, j);
        glVertex3f(i + 1, 0, j + 1);
        glVertex3f(i, 0, j + 1);
    }
}
glEnd();
glEnable(GL_LIGHTING);
glPopMatrix();

// Draw 3D playing field with depth box
glDisable(GL_LIGHTING);
glColor3f(0.3f, 0.5f, 0.7f);
glLineWidth(3.0f);

float depth = 3.0f;

glBegin(GL_LINES);
// Front face - main playing area
// Top wall
glVertex3f(-10, 7, 2);

```

```

glVertex3f(10, 7, 2);
// Bottom wall
glVertex3f(-10, -7, 2);
glVertex3f(10, -7, 2);
// Left side
glVertex3f(-10, -7, 2);
glVertex3f(-10, 7, 2);
// Right side
glVertex3f(10, -7, 2);
glVertex3f(10, 7, 2);

// Back face
glVertex3f(-10, 7, 2 - depth);
glVertex3f(10, 7, 2 - depth);
glVertex3f(-10, -7, 2 - depth);
glVertex3f(10, -7, 2 - depth);
glVertex3f(-10, -7, 2 - depth);
glVertex3f(-10, 7, 2 - depth);
glVertex3f(10, -7, 2 - depth);
glVertex3f(10, 7, 2 - depth);

// Connecting lines
glVertex3f(-10, 7, 2);
glVertex3f(-10, 7, 2 - depth);
glVertex3f(10, 7, 2);
glVertex3f(10, 7, 2 - depth);
glVertex3f(-10, -7, 2);
glVertex3f(-10, -7, 2 - depth);
glVertex3f(10, -7, 2);
glVertex3f(10, -7, 2 - depth);

// Center line (dashed)
glColor3f(0.5f, 0.6f, 0.8f);
for (int i = -6; i < 7; i++) {
    glVertex3f(0, i, 2);
    glVertex3f(0, i + 0.5f, 2);
}
glEnd();
glEnable(GL_LIGHTING);

```

```

// Player paddle - LEFT SIDE (stays in front)
glPushMatrix();
glTranslatef(-9, playerPaddleY, 0.5f);
glColor3f(0.0f, 1.0f, 1.0f);
glScalef(0.4f, 3.0f, 1.0f);
glutSolidCube(1.0f);

// Add glow
glDisable(GL_LIGHTING);
glColor3f(0.3f, 1.0f, 1.0f);
glutWireCube(1.1f);
glEnable(GL_LIGHTING);
glPopMatrix();

// AI paddle - RIGHT SIDE (stays in front)
glPushMatrix();
glTranslatef(9, aiPaddleY, 0.5f);
glColor3f(1.0f, 0.0f, 1.0f);
glScalef(0.4f, 3.0f, 1.0f);
glutSolidCube(1.0f);

// Add glow
glDisable(GL_LIGHTING);
glColor3f(1.0f, 0.3f, 1.0f);
glutWireCube(1.1f);
glEnable(GL_LIGHTING);
glPopMatrix();

// Ball (stays in playing field, not going into floor)
glPushMatrix();
glTranslatef(ballX, ballY, 0.5f);
glColor3f(1.0f, 1.0f, 0.0f);
glutSolidSphere(0.4f, 16, 16);

// Glow effect
glDisable(GL_LIGHTING);
glColor3f(1.0f, 0.8f, 0.0f);
glutWireSphere(0.5f, 12, 12);
glEnable(GL_LIGHTING);
glPopMatrix();

```

```

drawParticles();

char scoreText[50];
sprintf(scoreText, "Player: %d | AI: %d", playerScore, aiScore);
drawText(10, windowHeight - 30, scoreText);
drawText(10, 30, "Up/Down Arrows | ESC:Menu");

glutSwapBuffers();
}

// Shooter Game Functions
void initShooterGame() {
    playerX = 0;
    enemies.clear();
    bullets.clear();
    spawnCounter = 0;
    score = 0;
    lives = 3;
}

void updateShooterGame() {
    if (leftPressed) playerX -= 0.15f;
    if (rightPressed) playerX += 0.15f;
    if (playerX < -7) playerX = -7;
    if (playerX > 7) playerX = 7;

    spawnCounter++;
    if (spawnCounter > 60) {
        spawnCounter = 0;
        Enemy e;
        e.x = (rand() % 14) - 7.0f;
        e.z = -10;
        e.rotation = 0;
        enemies.push_back(e);
    }

    for (int i = enemies.size() - 1; i >= 0; i--) {
        enemies[i].z += 0.1f;
        enemies[i].rotation += 2.0f;
    }
}

```

```

if (enemies[i].z > 8) {
    lives--;
    playSoundEffect("explosion");
    createExplosion(enemies[i].x, 0, enemies[i].z);
    enemies.erase(enemies.begin() + i);
    if (lives <= 0) {
        if (score > highScores[2]) highScores[2] = score;
        previousGame = SHOOTER;
        currentGame = GAME_OVER;
        playSoundEffect("game_over");
        return;
    }
    continue;
}

if (fabs(enemies[i].x - playerX) < 1 && fabs(enemies[i].z - 0) < 1) {
    lives--;
    playSoundEffect("explosion");
    createExplosion(enemies[i].x, 0, enemies[i].z);
    enemies.erase(enemies.begin() + i);
    if (lives <= 0) {
        if (score > highScores[2]) highScores[2] = score;
        previousGame = SHOOTER;
        currentGame = GAME_OVER;
        playSoundEffect("game_over");
        return;
    }
}

}

for (int i = bullets.size() - 1; i >= 0; i--) {
    bullets[i].z -= 0.3f;

    if (bullets[i].z < -15) {
        bullets.erase(bullets.begin() + i);
        continue;
    }

    for (int j = enemies.size() - 1; j >= 0; j--) {

```

```

        if (fabs(bullets[i].x - enemies[j].x) < 1 &&
            fabs(bullets[i].z - enemies[j].z) < 1) {
            playSoundEffect("explosion");
            createExplosion(enemies[j].x, 0.5f, enemies[j].z);
            bullets.erase(bullets.begin() + i);
            enemies.erase(enemies.begin() + j);
            score += 10;
            playSoundEffect("score");
            break;
        }
    }
}

void drawShooterGame() {
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(45.0, (double>windowWidth / windowHeight, 1.0, 100.0);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
    gluLookAt(0, 8, 12, 0, 0, -2, 0, 1, 0);

    glEnable(GL_LIGHTING);
    drawGround();

    // Player ship
    glPushMatrix();
    glTranslatef(playerX, 0, 0);

    glColor3f(0.0f, 1.0f, 0.0f);
    glRotatef(-90, 1, 0, 0);
    glutSolidCone(0.5f, 1.0f, 4, 4);
    glRotatef(90, 1, 0, 0);

    glPushMatrix();
    glTranslatef(0, 0.3f, -0.3f);
    glRotatef(-90, 1, 0, 0);
    glColor3f(0.3f, 0.3f, 0.3f);

```

```

GLUquadricObj *quadric = gluNewQuadric();
gluCylinder(quadric, 0.1f, 0.1f, 0.6f, 8, 8);
gluDeleteQuadric(quadric);
glPopMatrix();

glPushMatrix();
glTranslatef(0, 0.4f, 0);
glColor3f(0.0f, 0.5f, 1.0f);
glutSolidSphere(0.2f, 12, 12);
glPopMatrix();

glPopMatrix();

// Enemies
for (int i = 0; i < enemies.size(); i++) {
    glPushMatrix();
    glTranslatef(enemies[i].x, 0.5f, enemies[i].z);
    glRotatef(enemies[i].rotation, 0, 1, 0);
    glRotatef(enemies[i].rotation * 0.5f, 1, 0, 0);
    glColor3f(1.0f, 0.0f, 0.0f);
    glutSolidCube(1.0f);

    glDisable(GL_LIGHTING);
    glColor3f(1.0f, 0.5f, 0.0f);
    glutWireCube(1.1f);
    glEnable(GL_LIGHTING);
    glPopMatrix();
}

// Bullets
for (int i = 0; i < bullets.size(); i++) {
    glPushMatrix();
    glTranslatef(bullets[i].x, 0, bullets[i].z);
    glColor3f(1.0f, 1.0f, 0.0f);
    glutSolidSphere(0.2f, 8, 8);

    glDisable(GL_LIGHTING);
    glColor3f(1.0f, 0.8f, 0.0f);
    glutWireSphere(0.25f, 6, 6);
    glEnable(GL_LIGHTING);
}

```

```

    glPopMatrix();
}

drawParticles();

char scoreText[50];
sprintf(scoreText, "Score: %d | Lives: %d | SPACE to Shoot!", score, lives);
drawText(10, windowHeight - 30, scoreText);
drawText(10, 30, "Left/Right Arrows + SPACE BAR | P:Pause | ESC:Menu");

glutSwapBuffers();
}

// Tetris Game Functions
bool tetrisCheck() {
    for (int i = 0; i < 4; i++) {
        if (tetrisCurrent[i].x < 0 || tetrisCurrent[i].x >= TETRIS_COLS ||
            tetrisCurrent[i].y >= TETRIS_ROWS)
            return false;
        if (tetrisBoard[tetrisCurrent[i].y][tetrisCurrent[i].x])
            return false;
    }
    return true;
}

void tetrisSpawnPiece() {
    tetrisCurrentType = tetrisNextType;
    tetrisCurrentColor = tetrisNextColor;
    tetrisNextType = rand() % 7;
    tetrisNextColor = 1 + rand() % 7;

    for (int i = 0; i < 4; i++) {
        tetrisCurrent[i].x = tetrisFigures[tetrisCurrentType][i] % 2 + TETRIS_COLS /
2 - 1;
        tetrisCurrent[i].y = tetrisFigures[tetrisCurrentType][i] / 2;
    }

    if (!tetrisCheck()) {
        if (score > highScores[3]) highScores[3] = score;
        previousGame = TETRIS;
    }
}

```

```

        currentGame = GAME_OVER;
        playSoundEffect("game_over");
    }
}

void tetrisRotate() {
    Point p = tetrisCurrent[1];
    for (int i = 0; i < 4; i++) {
        int x = tetrisCurrent[i].y - p.y;
        int y = tetrisCurrent[i].x - p.x;
        tetrisCurrent[i].x = p.x - x;
        tetrisCurrent[i].y = p.y + y;
    }
    if (!tetrisCheck()) {
        for (int i = 0; i < 4; i++)
            tetrisCurrent[i] = tetrisTemp[i];
    }
}

void initTetrisGame() {
    for (int i = 0; i < TETRIS_ROWS; i++) {
        for (int j = 0; j < TETRIS_COLS; j++) {
            tetrisBoard[i][j] = 0;
        }
    }
    tetrisNextType = rand() % 7;
    tetrisNextColor = 1 + rand() % 7;
    tetrisSpawnPiece();
    score = 0;
    tetrisLevel = 1;
    tetrisLinesCleared = 0;
    tetrisMoveCounter = 0;
    tetrisDx = 0;
    tetrisRotateFlag = false;
}

void updateTetrisGame() {
    tetrisMoveCounter++;
    int dropSpeed = 20 - (tetrisLevel * 2);
    if (dropSpeed < 5) dropSpeed = 5;

```

```

if (tetrisMoveCounter < dropSpeed) return;
tetrisMoveCounter = 0;

for (int i = 0; i < 4; i++) tetrisTemp[i] = tetrisCurrent[i];
for (int i = 0; i < 4; i++) tetrisCurrent[i].x += tetrisDx;
if (!tetrisCheck()) {
    for (int i = 0; i < 4; i++)
        tetrisCurrent[i] = tetrisTemp[i];
}

if (tetrisRotateFlag) tetrisRotate();

for (int i = 0; i < 4; i++) tetrisTemp[i] = tetrisCurrent[i];
for (int i = 0; i < 4; i++) tetrisCurrent[i].y += 1;

if (!tetrisCheck()) {
    for (int i = 0; i < 4; i++)
        tetrisBoard[tetrisTemp[i].y][tetrisTemp[i].x] = tetrisCurrentColor;
    score += 10;
    playSoundEffect("tetris_drop");
    tetrisSpawnPiece();
}

int k = TETRIS_ROWS - 1, linesRemoved = 0;
for (int i = TETRIS_ROWS - 1; i >= 0; i--) {
    int count = 0;
    for (int j = 0; j < TETRIS_COLS; j++)
        if (tetrisBoard[i][j]) count++;
    if (count < TETRIS_COLS) {
        for (int j = 0; j < TETRIS_COLS; j++)
            tetrisBoard[k][j] = tetrisBoard[i][j];
        k--;
    } else linesRemoved++;
}
while (k >= 0) {
    for (int j = 0; j < TETRIS_COLS; j++)
        tetrisBoard[k][j] = 0;
    k--;
}

```

```

if (linesRemoved > 0) {
    score += 100 * linesRemoved;
    tetrisLinesCleared += linesRemoved;
    if (linesRemoved >= 4) {
        playSoundEffect("line_clear"); // Special sound for Tetris!
    } else {
        playSoundEffect("score");
    }
    if (score >= tetrisLevel * 100) {
        tetrisLevel++;
    }
    for (int i = 0; i < linesRemoved * 5; i++) {
        createExplosion((rand() % TETRIS_COLS) * TETRIS_BLOCK_SIZE - 2,
            (rand() % 5) * TETRIS_BLOCK_SIZE,
            0);
    }
}

tetrisDx = 0;
tetrisRotateFlag = false;
}

void drawTetris3DBlock(float x, float y, float z, int c) {
    glPushMatrix();
    glTranslatef(x * TETRIS_BLOCK_SIZE, -y * TETRIS_BLOCK_SIZE, z *
TETRIS_BLOCK_SIZE);

    glColor3f(tetrisColors[c][0], tetrisColors[c][1], tetrisColors[c][2]);
    glutSolidCube(TETRIS_BLOCK_SIZE * 0.95f);

    glDisable(GL_LIGHTING);
    glColor3f(0.0f, 0.0f, 0.0f);
    glLineWidth(2.0f);
    glutWireCube(TETRIS_BLOCK_SIZE * 0.96f);
    glEnable(GL_LIGHTING);

    glPopMatrix();
}

void drawTetrisGame() {

```

```

glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

glMatrixMode(GL_PROJECTION);
glLoadIdentity();
gluPerspective(45.0, (double>windowWidth / windowHeight, 1.0, 100.0);
glMatrixMode(GL_MODELVIEW);
glLoadIdentity();

// Better camera angle for viewing the tetris board
float centerX = TETRIS_COLS * TETRIS_BLOCK_SIZE / 2.0f;
float centerY = -TETRIS_ROWS * TETRIS_BLOCK_SIZE / 2.0f;
gluLookAt(centerX, centerY + 5, 15, centerX, centerY, 0, 0, 1, 0);

glEnable(GL_LIGHTING);

// Draw floor under the tetris board
glPushMatrix();
glTranslatef(0, -TETRIS_ROWS * TETRIS_BLOCK_SIZE - 0.5f, 0);
glDisable(GL_LIGHTING);
glBegin(GL_QUADS);
for (int i = -10; i < 20; i++) {
    for (int j = -10; j < 10; j++) {
        if ((i + j) % 2 == 0)
            glColor3f(0.1f, 0.15f, 0.2f);
        else
            glColor3f(0.15f, 0.2f, 0.25f);

        glVertex3f(i * 0.5f, 0, j * 0.5f);
        glVertex3f((i + 1) * 0.5f, 0, j * 0.5f);
        glVertex3f((i + 1) * 0.5f, 0, (j + 1) * 0.5f);
        glVertex3f(i * 0.5f, 0, (j + 1) * 0.5f);
    }
}
glEnd();
glEnable(GL_LIGHTING);
glPopMatrix();

// Draw 3D grid box
glDisable(GL_LIGHTING);
glColor3f(0.4f, 0.45f, 0.5f);

```

```

glLineWidth(2.5f);

float depth = 2.0f;

glBegin(GL_LINES);

// Front face vertical and horizontal lines
for (int i = 0; i <= TETRIS_ROWS; i++) {
    glVertex3f(0, -i * TETRIS_BLOCK_SIZE, 0);
    glVertex3f(TETRIS_COLS * TETRIS_BLOCK_SIZE, -i *
TETRIS_BLOCK_SIZE, 0);
}
for (int i = 0; i <= TETRIS_COLS; i++) {
    glVertex3f(i * TETRIS_BLOCK_SIZE, 0, 0);
    glVertex3f(i * TETRIS_BLOCK_SIZE, -TETRIS_ROWS *
TETRIS_BLOCK_SIZE, 0);
}

// Left wall
for (int i = 0; i <= TETRIS_ROWS; i++) {
    glVertex3f(0, -i * TETRIS_BLOCK_SIZE, 0);
    glVertex3f(0, -i * TETRIS_BLOCK_SIZE, -depth);
}

// Right wall
for (int i = 0; i <= TETRIS_ROWS; i++) {
    glVertex3f(TETRIS_COLS * TETRIS_BLOCK_SIZE, -i *
TETRIS_BLOCK_SIZE, 0);
    glVertex3f(TETRIS_COLS * TETRIS_BLOCK_SIZE, -i *
TETRIS_BLOCK_SIZE, -depth);
}

// Bottom
for (int i = 0; i <= TETRIS_COLS; i++) {
    glVertex3f(i * TETRIS_BLOCK_SIZE, -TETRIS_ROWS *
TETRIS_BLOCK_SIZE, 0);
    glVertex3f(i * TETRIS_BLOCK_SIZE, -TETRIS_ROWS *
TETRIS_BLOCK_SIZE, -depth);
}

```

```

// Back edges
glVertex3f(0, 0, -depth);
glVertex3f(TETRIS_COLS * TETRIS_BLOCK_SIZE, 0, -depth);
glVertex3f(0, 0, -depth);
glVertex3f(0, -TETRIS_ROWS * TETRIS_BLOCK_SIZE, -depth);
glVertex3f(TETRIS_COLS * TETRIS_BLOCK_SIZE, 0, -depth);
glVertex3f(TETRIS_COLS * TETRIS_BLOCK_SIZE, -TETRIS_ROWS *
TETRIS_BLOCK_SIZE, -depth);
glVertex3f(0, -TETRIS_ROWS * TETRIS_BLOCK_SIZE, -depth);
glVertex3f(TETRIS_COLS * TETRIS_BLOCK_SIZE, -TETRIS_ROWS *
TETRIS_BLOCK_SIZE, -depth);

glEnd();
glEnable(GL_LIGHTING);

// Draw board blocks
for (int i = 0; i < TETRIS_ROWS; i++) {
    for (int j = 0; j < TETRIS_COLS; j++) {
        if (tetrisBoard[i][j]) {
            drawTetris3DBlock(j + 0.5f, i + 0.5f, 0, tetrisBoard[i][j]);
        }
    }
}

// Draw current falling piece
for (int i = 0; i < 4; i++) {
    if (tetrisCurrent[i].y >= 0 && tetrisCurrent[i].y < TETRIS_ROWS) {
        drawTetris3DBlock(tetrisCurrent[i].x + 0.5f, tetrisCurrent[i].y + 0.5f, 0,
tetrisCurrentColor);
    }
}

// Draw next piece preview box
float previewX = TETRIS_COLS * TETRIS_BLOCK_SIZE + 1.5f;
float previewY = -1.5f;

glDisable(GL_LIGHTING);
glColor3f(0.3f, 0.35f, 0.4f);
glLineWidth(2.0f);
glBegin(GL_LINE_LOOP);

```

```

glVertex3f(previewX - 0.5f, previewY - 0.5f, 0);
glVertex3f(previewX + 1.5f, previewY - 0.5f, 0);
glVertex3f(previewX + 1.5f, previewY - 2.5f, 0);
glVertex3f(previewX - 0.5f, previewY - 2.5f, 0);
glEnd();
glEnable(GL_LIGHTING);

glPushMatrix();
glTranslatef(previewX, previewY - 1.0f, 0);
for (int i = 0; i < 4; i++) {
    drawTetris3DBlock(tetrisFigures[tetrisNextType][i] % 2 + 0.2f,
        tetrisFigures[tetrisNextType][i] / 2 + 0.2f, 0, tetrisNextColor);
}
glPopMatrix();

drawParticles();

char scoreText[100];
sprintf(scoreText, "Score: %d | Level: %d | Lines: %d", score, tetrisLevel,
tetrisLinesCleared);
drawText(10, windowHeight - 30, scoreText);
drawText(10, 30, "Arrows: Move/Rotate | Down: Fast | SPACE: Drop | P:Pause |
ESC:Menu");

glutSwapBuffers();
}

// Display callback
void display() {
    if (currentGame == MENU) {
        drawMenu();
    } else if (currentGame == INSTRUCTIONS) {
        drawInstructions();
    } else if (currentGame == SNAKE) {
        drawSnakeGame();
    } else if (currentGame == PONG) {
        drawPongGame();
    } else if (currentGame == SHOOTER) {
        drawShooterGame();
    } else if (currentGame == TETRIS) {

```

```

        drawTetrisGame();
    } else if (currentGame == GAME_OVER) {
        drawGameOver();
    }
}

// Reshape callback
void reshape(int w, int h) {
    windowWidth = w;
    windowHeight = h;
    glViewport(0, 0, w, h);
}

// Keyboard callback
void keyboard(unsigned char key, int x, int y) {
    if (key == 27) { // ESC key
        if (currentGame != MENU && currentGame != INSTRUCTIONS) {
            currentGame = MENU;
        } else if (currentGame == INSTRUCTIONS) {
            currentGame = MENU;
        } else {
            exit(0);
        }
    }

    if (currentGame == MENU) {
        if (key == '1') {
            playSoundEffect("game_start");
            initSnakeGame();
            currentGame = SNAKE;
            previousGame = SNAKE;
        } else if (key == '2') {
            playSoundEffect("game_start");
            initPongGame();
            currentGame = PONG;
            previousGame = PONG;
        } else if (key == '3') {
            playSoundEffect("game_start");
            initShooterGame();
            currentGame = SHOOTER;
        }
    }
}

```

```

        previousGame = SHOOTER;
    } else if (key == '4') {
        playSoundEffect("game_start");
        initTetrisGame();
        currentGame = TETRIS;
        previousGame = TETRIS;
    } else if (key == 'i' || key == 'I') {
        playSoundEffect("menu_select");
        currentGame = INSTRUCTIONS;
    }
}

if (currentGame == INSTRUCTIONS && key == 13) { // Enter key
    playSoundEffect("menu_select");
    currentGame = MENU;
}

if (currentGame == GAME_OVER && key == 13) { // Enter key
    playSoundEffect("menu_select");
    currentGame = MENU;
}

if ((currentGame == SNAKE || currentGame == PONG || currentGame ==
SHOOTER || currentGame == TETRIS) &&
    (key == 'p' || key == 'P')) {
    playSoundEffect("pause");
    currentGame = PAUSED;
}

if (currentGame == SHOOTER && key == ' ') {
    playSoundEffect("shoot");
    Bullet b;
    b.x = playerX;
    b.z = -1;
    bullets.push_back(b);
}

if (currentGame == TETRIS && key == ' ') {
    playSoundEffect("tetris_drop");
}

```

```

while (true) {
    for (int i = 0; i < 4; i++) tetrisTemp[i] = tetrisCurrent[i];
    for (int i = 0; i < 4; i++) tetrisCurrent[i].y += 1;
    if (!tetrisCheck()) {
        for (int i = 0; i < 4; i++)
            tetrisBoard[tetrisTemp[i].y][tetrisTemp[i].x] = tetrisCurrentColor;
        score += 10;
        tetrisSpawnPiece();
        break;
    }
}
}
}

```

```

void keyboardUp(unsigned char key, int x, int y) {
    // Handle key releases if needed
}

```

// Special keys callback

```

void specialKeys(int key, int x, int y) {
    if (currentGame == SNAKE) {
        if (key == GLUT_KEY_UP && snakeDir != 3) nextDir = 1;
        else if (key == GLUT_KEY_DOWN && snakeDir != 1) nextDir = 3;
        else if (key == GLUT_KEY_LEFT && snakeDir != 0) nextDir = 2;
        else if (key == GLUT_KEY_RIGHT && snakeDir != 2) nextDir = 0;
    }

    if (currentGame == PONG) {
        if (key == GLUT_KEY_UP) upPressed = true;
        else if (key == GLUT_KEY_DOWN) downPressed = true;
    }

    if (currentGame == SHOOTER) {
        if (key == GLUT_KEY_LEFT) leftPressed = true;
        else if (key == GLUT_KEY_RIGHT) rightPressed = true;
    }

    if (currentGame == TETRIS) {
        if (key == GLUT_KEY_LEFT) tetrisDx = -1;
        else if (key == GLUT_KEY_RIGHT) tetrisDx = 1;
    }
}

```

```

        else if (key == GLUT_KEY_UP) tetrisRotateFlag = true;
        else if (key == GLUT_KEY_DOWN) tetrisMoveCounter = 100;
    }
}

void specialKeysUp(int key, int x, int y) {
    if (currentGame == PONG) {
        if (key == GLUT_KEY_UP) upPressed = false;
        else if (key == GLUT_KEY_DOWN) downPressed = false;
    }

    if (currentGame == SHOOTER) {
        if (key == GLUT_KEY_LEFT) leftPressed = false;
        else if (key == GLUT_KEY_RIGHT) rightPressed = false;
    }
}

// Timer callback
void timer(int value) {
    gameTime += 0.016f;

    if (currentGame == SNAKE) updateSnakeGame();
    else if (currentGame == PONG) updatePongGame();
    else if (currentGame == SHOOTER) updateShooterGame();
    else if (currentGame == TETRIS) updateTetrisGame();

    updateParticles();

    glutPostRedisplay();
    glutTimerFunc(16, timer, 0);
}

// Main function
int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(windowWidth, windowHeight);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("Enhanced 3D Game Collection with Tetris - OpenGL
    GLUT");
}

```

```

init();

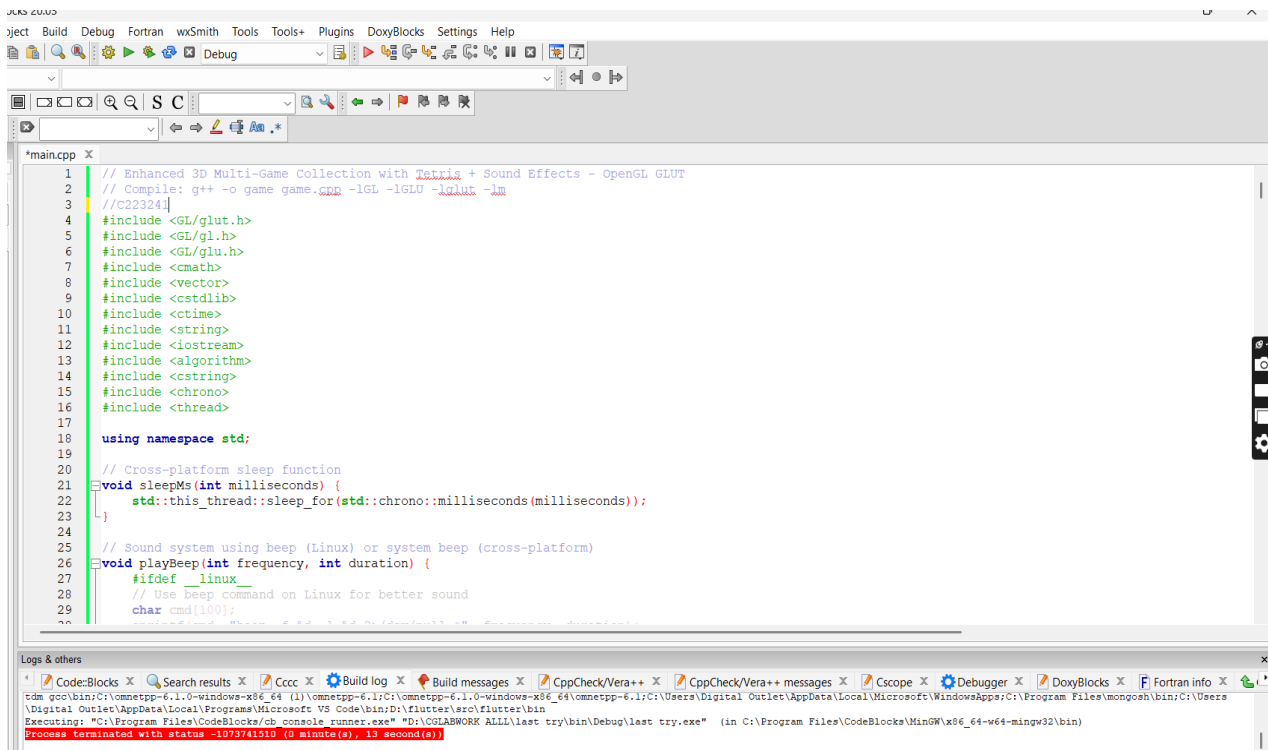
glutDisplayFunc(display);
glutReshapeFunc(reshape);
glutKeyboardFunc(keyboard);
glutKeyboardUpFunc(keyboardUp);
glutSpecialFunc(specialKeys);
glutSpecialUpFunc(specialKeysUp);
glutTimerFunc(0, timer, 0);

cout << "===== " << endl;
cout << "  Your ultimate destination for endless fun" << endl;
cout << "===== " << endl;
cout << "Press 1: Snake Game" << endl;
cout << "Press 2: Pong Game" << endl;
cout << "Press 3: Shooter Game" << endl;
cout << "Press 4: Tetris Game" << endl;
cout << "Press I: Instructions" << endl;
cout << "Press ESC: Back to menu / Quit" << endl;
cout << "===== " << endl;

glutMainLoop();
return 0;
}

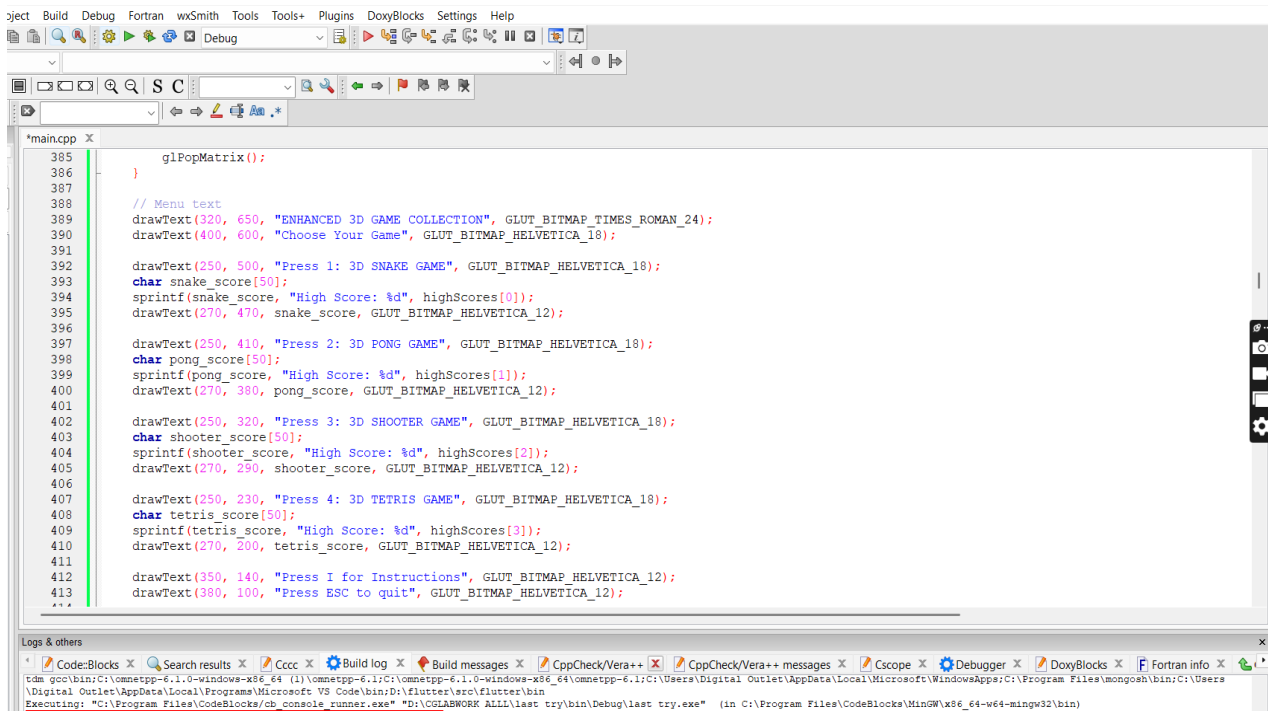
```

7. Code Screenshot:



```
*main.cpp x
1 // Enhanced 3D Multi-Game Collection with Tetris + Sound Effects - OpenGL GLUT
2 // Compile: g++ -o game game.cpp -lGL -lGLU -lglut -lm
3 //C223241
4 #include <GL/glut.h>
5 #include <GL/glu.h>
6 #include <GL/glut.h>
7 #include <cmath>
8 #include <vector>
9 #include <cstdlib>
10 #include <ctime>
11 #include <string>
12 #include <iostream>
13 #include <algorithm>
14 #include <string>
15 #include <chrono>
16 #include <thread>
17
18 using namespace std;
19
20 // Cross-platform sleep function
21 void sleepMs(int milliseconds) {
22     std::this_thread::sleep_for(std::chrono::milliseconds(milliseconds));
23 }
24
25 // Sound system using beep (Linux) or system beep (cross-platform)
26 void playBeep(int frequency, int duration) {
27     #ifdef _linux_
28         // Use beep command on Linux for better sound
29         char cmd[100];
30         sprintf(cmd, "beep -f %d -l %d", frequency, duration);
31         system(cmd);
32     #endif
33 }
```

Fig:1. C++ Code.



```
*main.cpp x
385 glPopMatrix();
386 }
387
388 // Menu text
389 drawText(320, 650, "ENHANCED 3D GAME COLLECTION", GLUT_BITMAP_TIMES_ROMAN_24);
390 drawText(400, 600, "Choose Your Game", GLUT_BITMAP_HELVETICA_18);
391
392 drawText(250, 500, "Press 1: 3D SNAKE GAME", GLUT_BITMAP_HELVETICA_18);
393 char snake_score[50];
394 sprintf(snake_score, "High Score: %d", highScores[0]);
395 drawText(270, 470, snake_score, GLUT_BITMAP_HELVETICA_12);
396
397 drawText(250, 410, "Press 2: 3D PONG GAME", GLUT_BITMAP_HELVETICA_18);
398 char pong_score[50];
399 sprintf(pong_score, "High Score: %d", highScores[1]);
400 drawText(270, 380, pong_score, GLUT_BITMAP_HELVETICA_12);
401
402 drawText(250, 320, "Press 3: 3D SHOOTER GAME", GLUT_BITMAP_HELVETICA_18);
403 char shooter_score[50];
404 sprintf(shooter_score, "High Score: %d", highScores[2]);
405 drawText(270, 290, shooter_score, GLUT_BITMAP_HELVETICA_12);
406
407 drawText(250, 230, "Press 4: 3D TETRIS GAME", GLUT_BITMAP_HELVETICA_18);
408 char tetris_score[50];
409 sprintf(tetris_score, "High Score: %d", highScores[3]);
410 drawText(270, 200, tetris_score, GLUT_BITMAP_HELVETICA_12);
411
412 drawText(350, 140, "Press I for Instructions", GLUT_BITMAP_HELVETICA_12);
413 drawText(380, 100, "Press ESC to quit", GLUT_BITMAP_HELVETICA_12);
414 }
```

Fig:2. C++ Code (contd.).

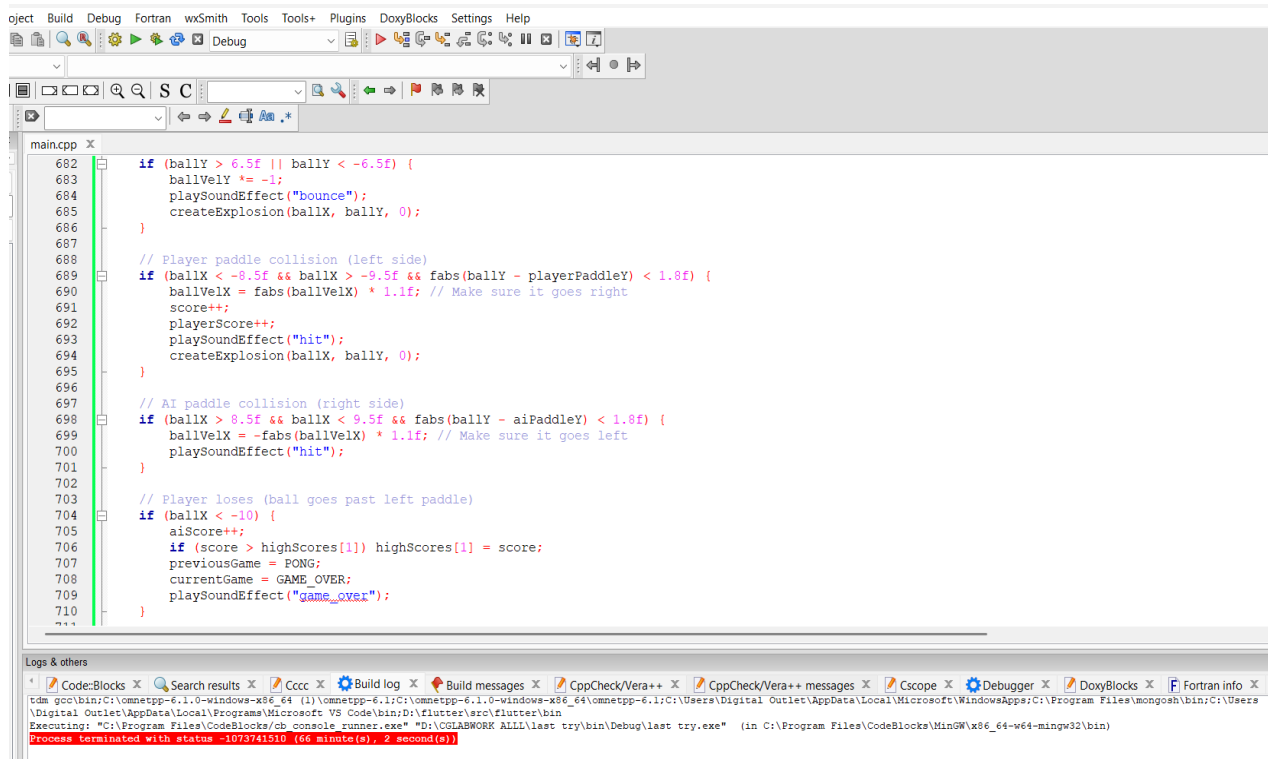


Fig:3. C++ Code (contd.).

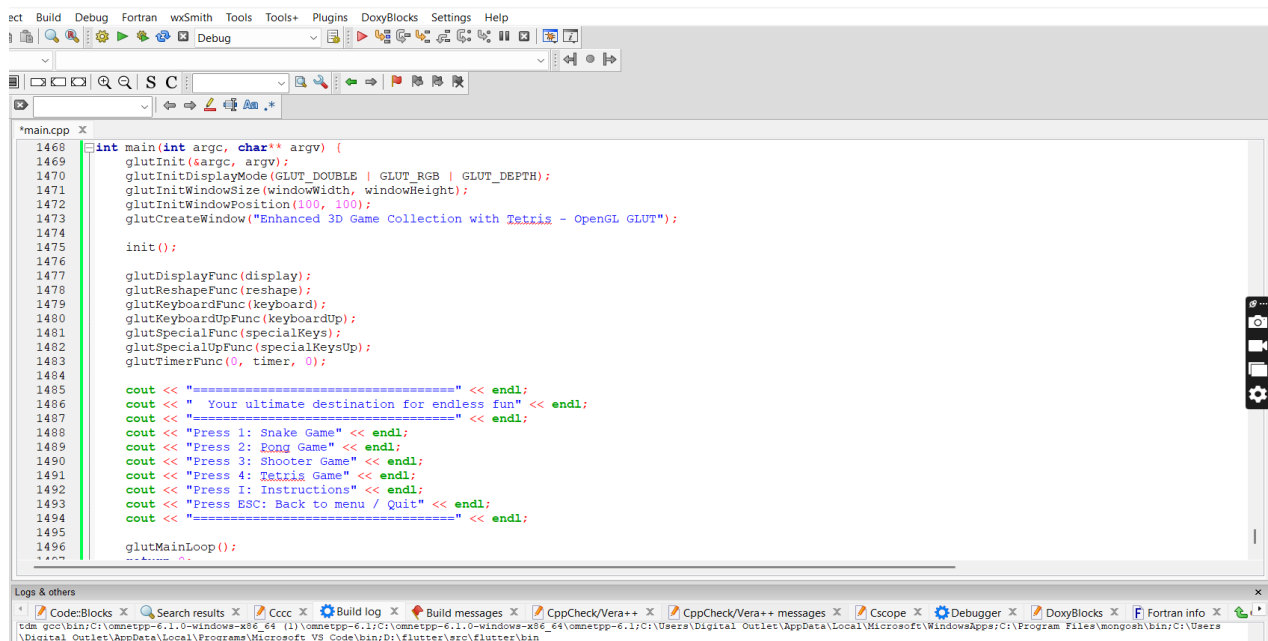


Fig:4. C++ Code (contd.).

8.Result Screenshot:

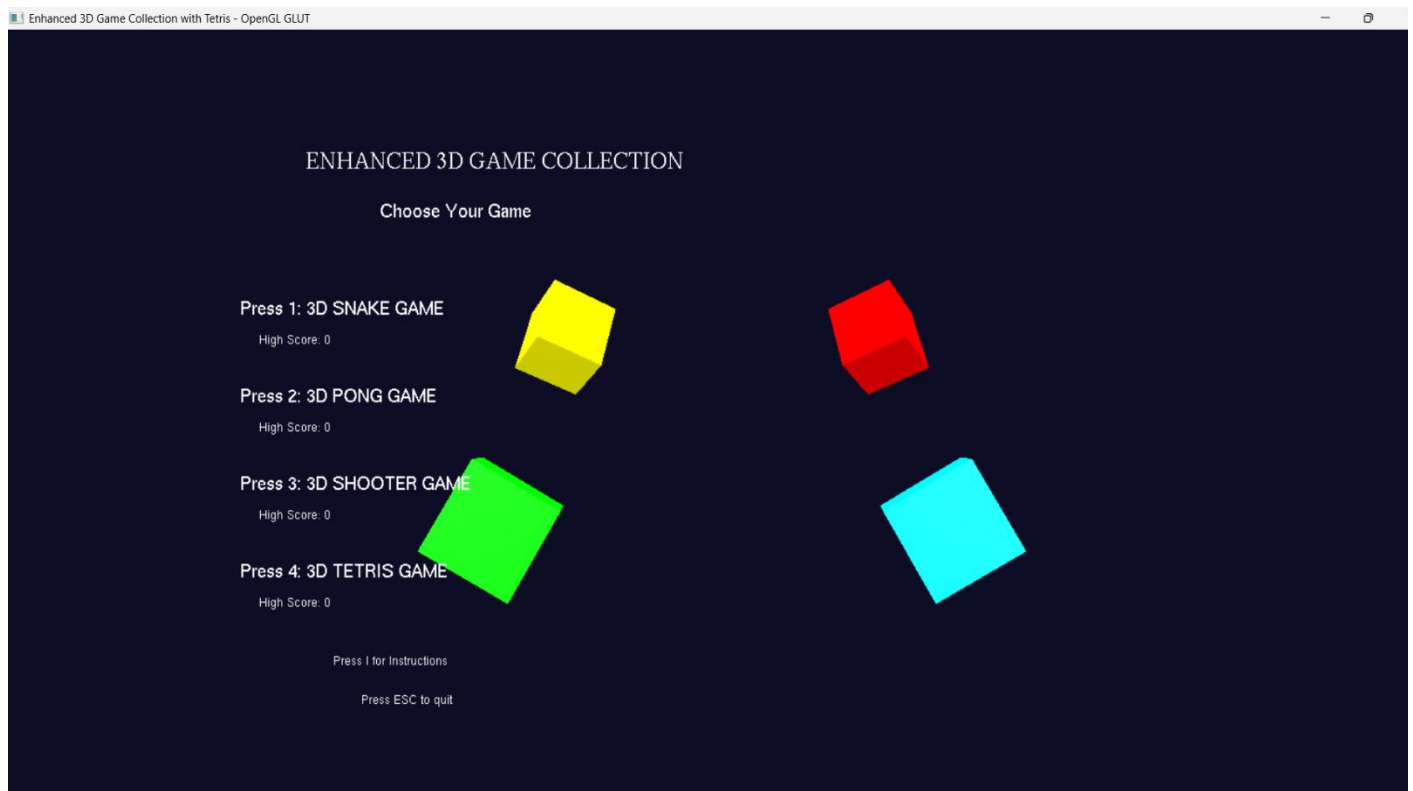


Fig:5. Game Startup Interface.

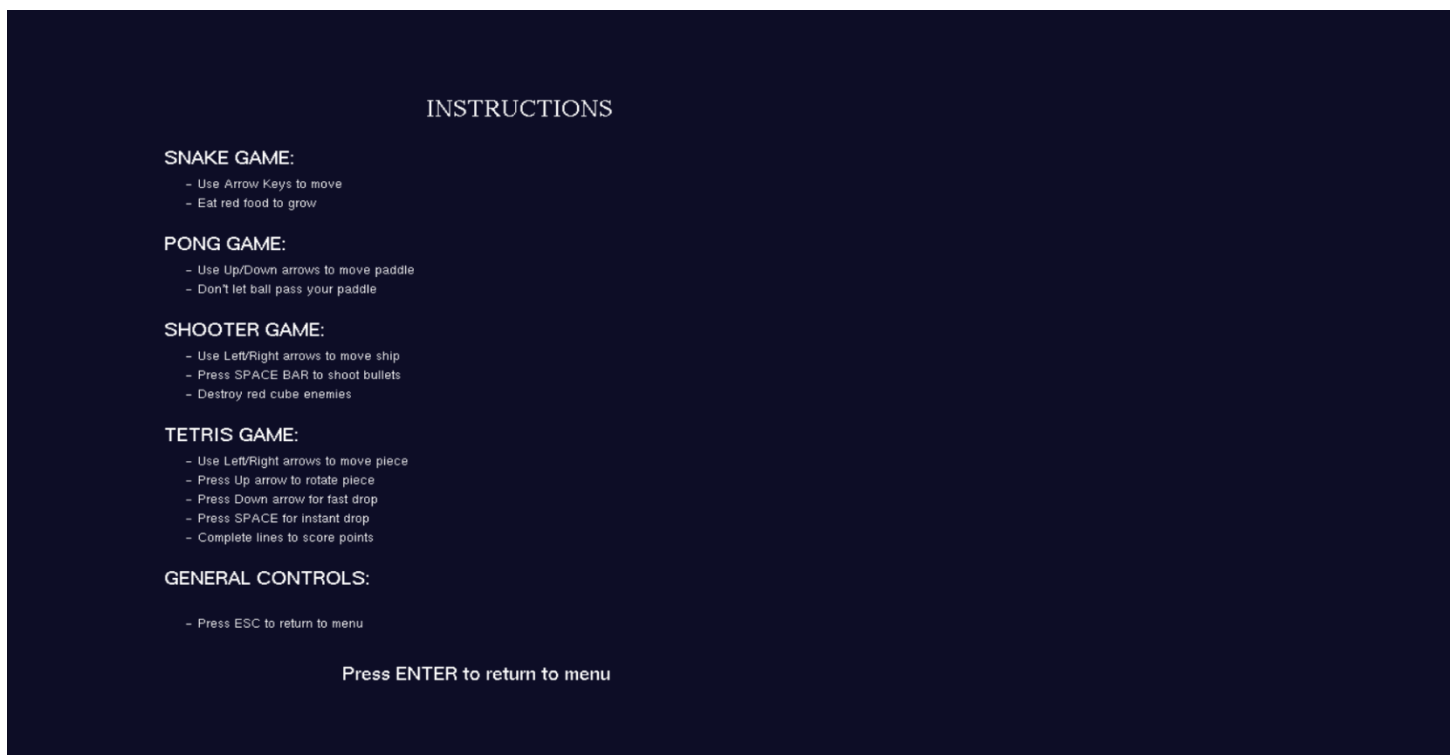


Fig:6. Game Instruction Interface.



Fig:7. 3D Snake Game Interface.

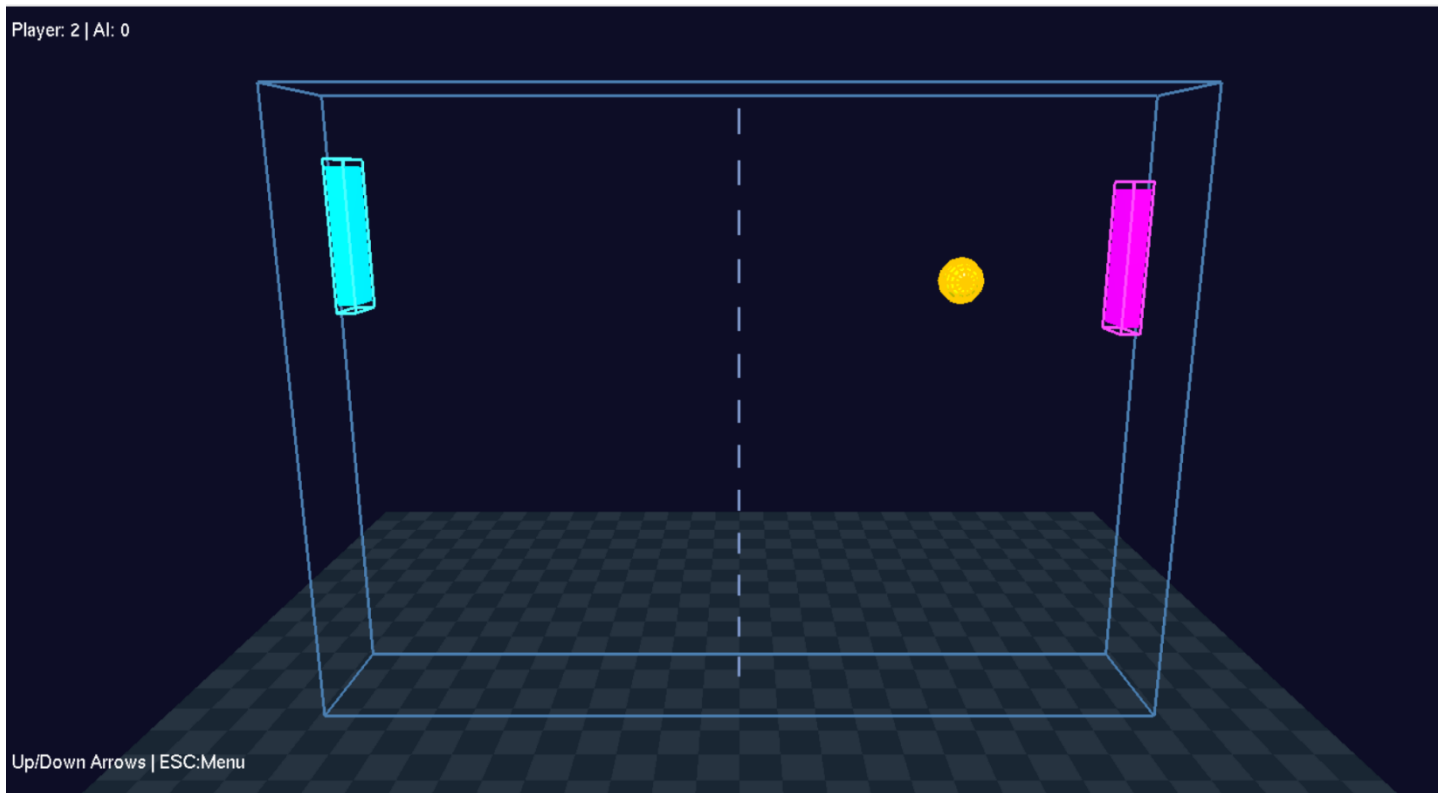


Fig:8. 3D Pong Game Interface.

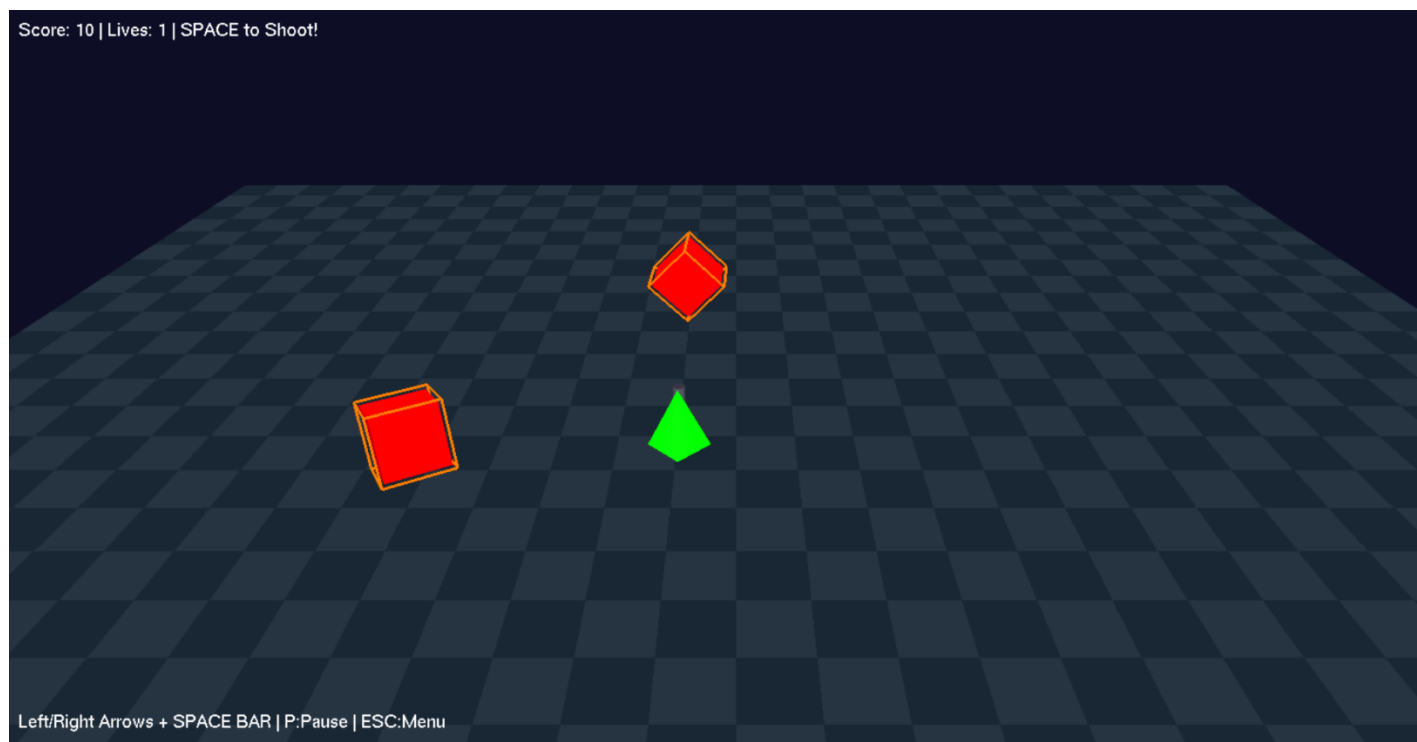


Fig:9. 3D Shooter Game Interface.

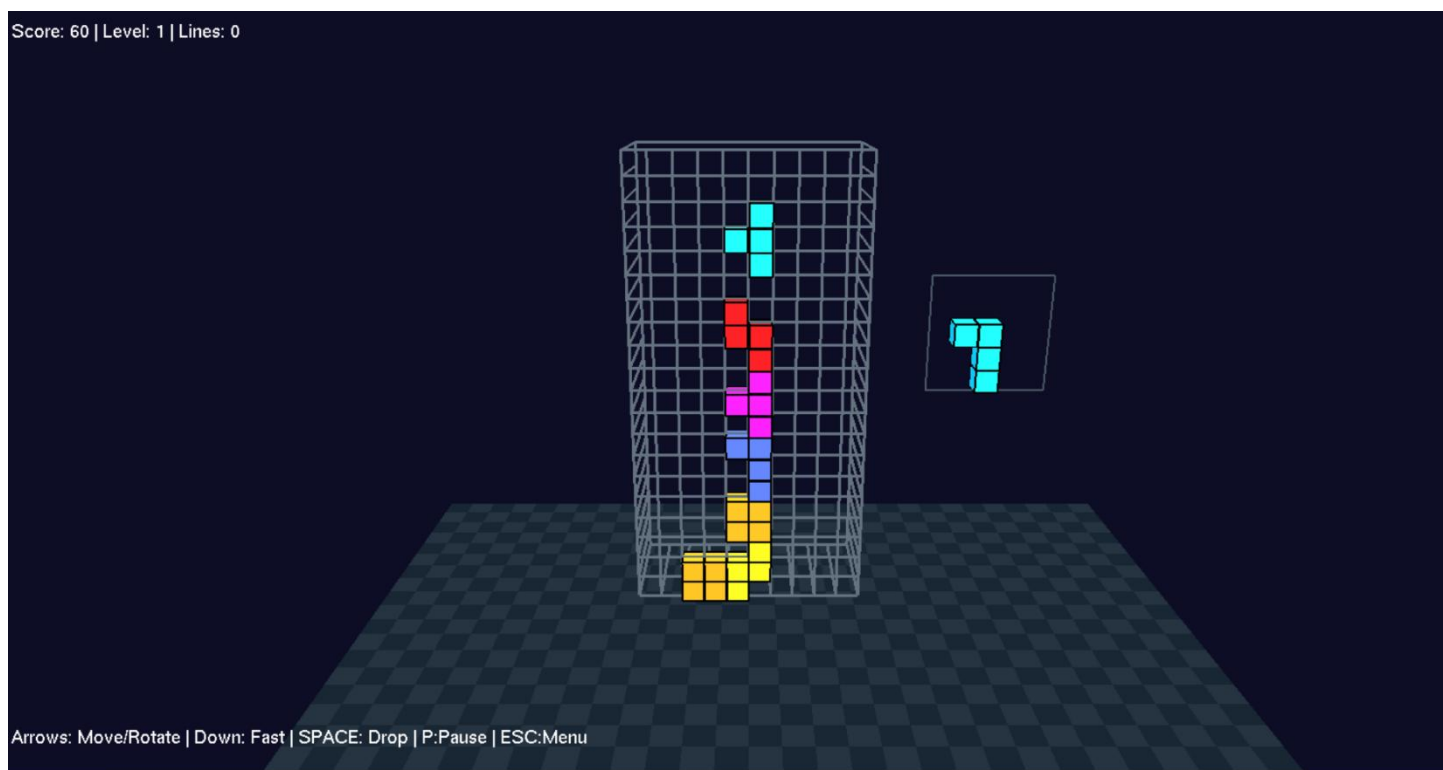


Fig:10. 3D Tetris Game Interface.

9. Applications

- Educational tool for **computer graphics learning**
- Game development practice project
- Demonstration of **OpenGL rendering techniques**
- Suitable for **graphics lab and academic submission**

10. Future Scope

- Add more games to the platform
- Implement multiplayer mode
- Add texture mapping and animations
- Use real sound files instead of basic sounds
- Online leaderboard system
- Mobile or VR version support

11. Conclusion

The project of multiple 3D Game Collection project successfully demonstrates the use of OpenGL and GLUT in game development. The Snake and Tetris games were implemented with proper game logic and graphics rendering. This project improves understanding of computer graphics, real-time interaction, and game design concepts, making it suitable as an academic and practical project.

Reference:



Snake Game Demo

<https://youtube.com/shorts/r5vzB8M-Q28?si=w0tg4Bq3k8YDC3gP>



Tetris Game Demo

<https://youtube.com/shorts/-ZgvgwbbQiM?si=k3TVvzLJDWuEWN7p>